# Software Requirements Specification (SRS)

## Cravify – Full-Stack Food Delivery Platform

202512030- ayush soni

202512098- parin makwana

202512042- hitarth shah

202512009- nevil nandasana

# 1. Introduction

## 1.1. Purpose

This document outlines the comprehensive specifications for Cravify, a full-stack food delivery platform that connects customers with restaurants and delivery partners. The system enables users to browse restaurants, order food, track deliveries in real-time, and make secure payments while providing restaurant vendors and delivery partners with efficient tools to manage their operations.

## 1.2. Scope

- Customer registration
- Restaurant and menu browsing with detailed information
- Cart management and order placement
- Real-time order tracking and status updates
- Multi-channel payment integration
- Restaurant vendor dashboard for menu and order management
- Administrative panel for platform oversight
- Rating and review system

## 1.3. Intended Audience

This document is intended for project stakeholders, development team members, quality assurance testers, UI/UX designers, database administrators, operations engineers, and maintenance personnel involved in the Cravify platform lifecycle.

## 1.4. Product Features

### 1.4.1. Customer Module

- User registration and authentication with mobile number/email
- Profile management with saved addresses and preferences
- Browse restaurant menus with item details and images
- Shopping cart management (single restaurant per order)
- Multiple payment options (UPI, cards, net banking, Cash on Delivery)
- Order history with reorder functionality
- Restaurant and delivery partner ratings and reviews
- Coupon application and discount management
- Customer support integration

### 1.4.2. Restaurant Vendor Module

- Admin approval workflow for restaurant onboarding
- Comprehensive menu management (add, edit, delete items)
- Item availability and preparation time settings
- Order acceptance and rejection capabilities
- Order status management (preparing, ready for pickup)
- Order history and analytics

- Customer feedback and ratings visibility
- Operating hours and restaurant availability management

#### 1.4.3. Admin Module

- Comprehensive user management (customers, restaurants, delivery partners)
- Restaurant approval and verification workflow
- Delivery partner approval and document verification
- Platform-wide order monitoring and analytics
- Manual order cancellation and dispute resolution
- User account suspension and blocking capabilities
- Real-time dashboard with key metrics
- Performance analytics (order volume, delivery times, ratings)
- Promotional campaign management
- System configuration and settings management

## 1.5. Non-Functional Requirements

| Requirement Category | Specification |
|---|---|
| **Performance** | Application should load in under some seconds. API response time < 500ms for 95% of requests. Support 1000+ concurrent users without degradation. |
| **Security & Privacy** | HTTPS encryption for all communications. Secure password storage with bcrypt hashing. |
| **Scalability** | Horizontal scaling capability for increased load. Database connection pooling and optimization. CDN integration for static content delivery. Load balancer configuration for traffic distribution. |
| **Maintainability** | Modular codebase with clear separation of concerns. Comprehensive API documentation. Automated testing with >70% code coverage. Logging and monitoring integration. |
| **Usability** | Intuitive UI/UX following platform guidelines. Responsive design for multiple screen sizes. |

## 1.6. Architecture Overview

**Technology Stack:**

| Component | Technology |
|---|---|
| **Frontend (Web)** | React.js with modern JavaScript , HTML5, CSS3 |
| **Frontend (Mobile)** | React Native for cross-platform development |
| **Backend** | Node.js with Express |
| **Database** | MySQL alternative, MongoDB |
| **Deployment** | Cloud platform (AWS/GCP/Azure) or self-hosted (Render/Railway) |

## 1.7. Future Enhancements

- Social features (share orders, group ordering)
- Subscription models for premium users
- Restaurant loyalty programs
- Live chat support with automated chatbots
- Predictive delivery time estimation using historical data
- Multi-language support for diverse user base
- Integration with food safety and hygiene ratings

## 1.8. User Stories for Cravify Platform

| ID | User Role | Wants To | So That |
|---|---|---|---|
| **US001** | Customer | register using mobile number or email | I can securely access the platform |
| **US002** | Customer | log in with OTP or password | I can quickly access my account |
| **US003** | Customer | search and filter restaurants by cuisine, rating, or delivery time | I can find food that matches my preferences |
| **US004** | Customer | view restaurant menus with item details and prices | I can make informed ordering decisions |
| **US005** | Customer | add items to cart from a single restaurant | I can prepare my order before checkout |
| **US006** | Customer | apply discount coupons at checkout | I can save money on my order |
| **US007** | Customer | choose from multiple payment methods | I can pay using my preferred option |
| **US008** | Customer | track my order in real-time | I know exactly when my food will arrive |
| **US009** | Customer | receive notifications for order status changes | I stay updated without checking the app constantly |
| **US010** | Customer | view my order history | I can reorder my favorite meals easily |
| **US011** | Customer | rate and review restaurants and delivery partners | I can share my experience and help others |
| **US012** | Customer | save multiple delivery addresses | I can order to different locations conveniently |
| **US013** | Restaurant Owner | register my restaurant with complete details | customers can discover my business |
| **US014** | Restaurant Owner | manage my menu by adding, editing, or removing items | my offerings stay current |
| **US015** | Restaurant Owner | mark items as unavailable temporarily | customers only see what I can prepare |
| **US016** | Restaurant Owner | accept or reject incoming orders | I can manage my kitchen workload |
| **US017** | Restaurant Owner | update order preparation status | customers and delivery partners know when food is ready |
| **US018** | Restaurant Owner | view customer ratings and reviews | I can improve my service quality |

| | | | |
|---|---|---|---|
| **US019** | Restaurant Owner | access order history and analytics | I can track my business performance |
| **US020** | Delivery Partner | register with my vehicle and license details | I can work as a delivery partner |
| **US021** | Delivery Partner | receive order assignment notifications | I can accept delivery requests |
| **US022** | Delivery Partner | view pickup location and delivery address | I can plan my route efficiently |
| **US023** | Delivery Partner | update delivery status at each stage | customers track their order accurately |
| **US024** | Delivery Partner | view my delivery history and earnings | I can track my income |
| **US025** | Admin | approve restaurant registrations after verification | only legitimate businesses join the platform |
| **US026** | Admin | approve delivery partner applications | verified partners deliver orders |
| **US027** | Admin | monitor all orders across the platform | I can identify and resolve issues |

## 1.9. Cravify Application - Project Plan

### Overview

- Project Duration: Approximately 15-18 weeks
- Development Methodology: Agile (Scrum-based with 2-week sprints)
- Team Size: 4 developers with distributed module ownership
- Deployment Strategy: Progressive rollout with MVP launch

### Phase 1: Requirement Analysis & Planning (Week 1)

**Deliverables:**

- Finalized Software Requirements Specification (SRS)
- Prioritized feature backlog with story points
- Technology stack finalized with justification
- Project roadmap and Gantt chart
- Team roles and responsibilities matrix

**Tasks:**

- Conduct stakeholder meetings to gather detailed requirements
- Define MVP scope and post-launch features
- Create database schema
- Establish coding standards and review processes

## Phase 2: Design (Week 2-3)

**Deliverables:**

- UI/UX wireframes and high-fidelity mockups
- Database ER diagram and schema design
- API specification document with endpoints
- System architecture diagram
- User flow diagrams for all modules

**Tasks:**

- Create wireframes for all user interfaces (customer, restaurant, delivery, admin)
- Design mockups in Figma with style guide
- Define RESTful API contracts
- Get stakeholder approval on designs
- Set up frontend and backend project repositories

## Phase 3: Backend Development (Week 4-7)

**Deliverables:**

- REST APIs for all modules (authentication, orders, restaurants, delivery)

- Database implementation with PostgreSQL
- Payment gateway integration (mock for development)
- Real-time notification service
- Initial unit tests for backend services

**Tasks:**

- Set up Node.js project structure
- Implement user authentication and authorization
- Build APIs for customer, restaurant, delivery, and admin modules
- Integrate payment gateway with test environment
- Implement order management workflow
- Set up database migrations and seeders
- Configure CI/CD pipeline basics

## Phase 4: Frontend Web Development (Week 8-11)

**Deliverables:**

- Customer web application (React)
- Restaurant dashboard
- Admin panel
- Responsive design implementation
- API integration complete

**Tasks:**

- Develop customer-facing pages (home, restaurant listing, menu, checkout)
- Create admin panel with analytics
- Integrate frontend with backend APIs
- Implement state management
- Add form validations and error handling
- Ensure responsive design for mobile browsers

## Phase 5: Integration Testing & Quality Assurance (Week 12-13)

**Deliverables:**

- Comprehensive test cases document
- Integration test results
- Bug tracking and resolution log
- Performance testing report
- Security audit findings

**Tasks:**

- Conduct end-to-end testing of all user workflows

- Perform API testing using Postman/Newman
- Load testing for concurrent users
- Cross-browser and cross-device testing
- Bug fixing and regression testing

## Phase 6: DevOps & Deployment (Week 14-15)

**Deliverables:**

- Production environment setup on cloud platform
- Docker containerization of services
- CI/CD pipeline for automated deployment
- Monitoring and logging dashboards
- Database backup and recovery procedures

**Tasks:**

- Set up production infrastructure
- Configure SSL certificates for HTTPS
- Implement database backup automation
- Deploy backend and frontend to production
- Set up CDN for static assets

## Phase 7: MVP Launch (Week 16-18)

**Deliverables:**

- Live customer web application
- Restaurant and delivery partner onboarding complete
- Admin dashboard operational
- Marketing materials and launch campaign

**Tasks:**

- Final production deployment
- App store submission and approval
- Onboard pilot restaurants and delivery partners
- Train customer support team
- Launch marketing campaign
- Monitor system performance and user feedback
- Prepare for post-launch support and iterations

## Tools & Technologies

| Area | Tools / Stack |
|---|---|
| **Frontend (Web)** | React.js, HTML5, CSS3, JavaScript (ES6+) |
| **Backend** | Spring Boot (Java) / Node.js with Express |
| **Database** | MySQL, MongoDB |
| **DevOps** | Docker, AWS/GCP, Jenkins/GitHub Actions |
| **Design** | Figma, Adobe XD |
| **Communication** | Google Meet, Discord |

**Testing Tools:**

- API Testing: Postman, Insomnia
- Load Testing: JMeter, Artillery

## Timeline Overview

| Phase | Duration (Weeks) | Sprint(s) |
|---|---|---|
| **Req. Analysis & Planning** | 1 | Sprint 1 |
| **Design** | 2 | Sprint 2 |
| **Backend Development** | 4 | Sprint 3-4 |
| **Frontend Web Development** | 4 | Sprint 5-6 |
| **Testing & QA** | 1-2 | Sprint 7 |
| **DevOps & Deployment** | 1-2 | Sprint 8 |
| **MVP Launch** | 1-2 | Sprint 9 |

# 2. Team Member Work Distribution

## 2.1. Team Member 1 - Customer Module (Ayush Soni)

**Primary Responsibilities:**

- Customer authentication system (registration, login)
- User profile management and address handling
- Restaurant listing and search functionality
- Restaurant and menu browsing interface
- Shopping cart implementation
- Checkout flow and order placement
- Order tracking interface
- Order history display
- Rating and review system for customers
- Customer-side API integration

## 2.2. Team Member 2 - Restaurant Vendor Module (Parin Makwana)

**Primary Responsibilities:**

- Restaurant registration and profile setup
- Restaurant dashboard interface
- Menu management system (CRUD operations)
- Item availability and preparation time settings
- Incoming order notification system
- Order acceptance/rejection workflow
- Order status update functionality
- Restaurant order history and analytics
- Vendor feedback and ratings display
- Restaurant-side API integration

## 2.3. Team Member 3 - Delivery Partner Module (Nevil Nandasana)

**Primary Responsibilities:**

- Delivery partner registration and verification
- Delivery partner mobile application
- Order assignment and notification system
- Accept/reject delivery request interface
- GPS navigation and route optimization
- Multi-stage delivery status updates
- Delivery history tracking
- Earnings calculation and summary
- Availability toggle (online/offline)
- Delivery partner API integration

## 2.4. Team Member 4 - Admin Module (Hitarth Shah)

**Primary Responsibilities:**

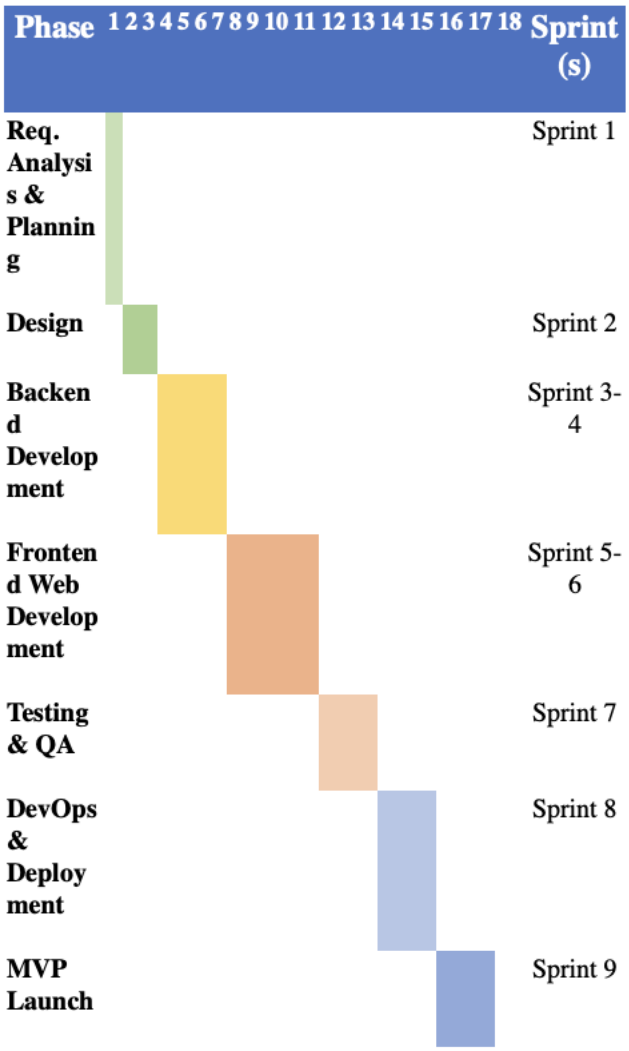
- Admin authentication and access control
- User management interface (view, block, unblock)
- Restaurant approval workflow
- Delivery partner verification and approval
- Platform-wide order monitoring dashboard
- Manual order cancellation capability
- Analytics and reporting system
- Promotional campaign management
- Platform configuration and settings
- Admin panel API integration

## 3. Database Design

### 3.1. Relationships

The database follows a relational model with proper foreign key constraints. Key relationships include: One-to-Many between Restaurants and MenuItems, Many-to-Many between Orders and MenuItems (with OrderItems junction table), One-to-Many between Users and Orders, One-to-One between Orders and DeliveryAssignments, and Many-to-One between Reviews and Users/Restaurants.

## Gant chart:-



## Legend

| Phase | Color Code |
|---|---|
| Req. Analysis & Planning | Light Green |
| Design | Medium Green |
| Backend Development | Yellow |
| Frontend Web Development | Orange |
| Testing & QA | Light Orange |
| DevOps & Deployment | Light Blue |
| MVP Launch | Medium Blue |

## Detailed Timeline Summary

| Phase | Weeks | Duration | Sprint(s) |
|---|---|---|---|
| **Req. Analysis & Planning** | Week 1 | 1 week | Sprint 1 |
| **Design** | Week 2-3 | 2 weeks | Sprint 2 |
| **Backend Development** | Week 4-7 | 4 weeks | Sprint 3-4 |
| **Frontend Web Development** | Week 8-11 | 4 weeks | Sprint 5-6 |
| **Testing & QA** | Week 12-13 | 2 weeks | Sprint 7 |
| **DevOps & Deployment** | Week 14-15 | 2 weeks | Sprint 8 |
| **MVP Launch** | Week 16-18 | 2 weeks | Sprint 9 |

### Sprint Breakdown (2-Week Sprints)

| Sprint | Weeks | Focus Area |
|---|---|---|
| **Sprint 1** | Week 1-2 | Requirements & Design Start |
| **Sprint 2** | Week 3-4 | Design Complete & Backend Start |
| **Sprint 3** | Week 5-6 | Backend Development - Core APIs |
| **Sprint 4** | Week 7-8 | Backend Complete & Frontend Start |
| **Sprint 5** | Week 9-10 | Frontend Development - Customer & Restaurant |
| **Sprint 6** | Week 11-12 | Frontend Complete & Testing Start |
| **Sprint 7** | Week 13-14 | Testing & DevOps Start |
| **Sprint 8** | Week 15-16 | DevOps Complete & Launch Prep |
| **Sprint 9** | Week 17-18 | MVP Launch & Monitoring |

## 4. Conclusion

This Software Requirements Specification provides a comprehensive blueprint for developing Cravify, a full-stack food delivery platform. The project is structured across 12 weeks with clear phases, deliverables, and team responsibilities. By following Agile methodology and maintaining focus on quality, security, and user experience, the team will deliver a robust MVP that meets stakeholder expectations and serves as a foundation for future enhancements.